



Módulo 3 · Testing, Debugging y Documentación con IA

Pruebas Unitarias con IA — Fundamentos y Estructura del Prompt

Ejercicio de Clase · Ejercicio 1 de 2 · Bloques 1.1 — 1.2 — 1.3 — 1.4

Nombre del alumno/a	Angel Echenique
Fecha	22/03/2026
🔗 Enlace GitHub del repositorio	

🔗 **Instrucciones:** Responde con lo que hemos visto en clase. Puedes consultar tus apuntes. No hay que escribir código, ¡solo demostrar que entendiste!

BLOQUE 1 — ¿Por qué usar IA para testing? (1.1)

En clase vimos **6 ventajas clave** de usar IA para generar tests. Escríbelas y explica brevemente para qué sirve cada una:

Nº	Nombre de la ventaja	¿Para qué sirve concretamente?
1	Genera test rapidamente	Crea las prueba en segundo y solo tenemos que revisarlas
2	Sugiere casos edge	Detecta situaciones raras que nosotros olvidamos a veces.
3	Crea mock y stubs automaticamente	Simula bases de datos o APIs sin necesitarlas de verdad.

4	Aumenta la cobertura	Prueba más caminos del código (errores, casos raros, etc.)
5	Reduce errores humanos	Evita que olvides tests o cometas fallos básicos.
6	Enseña buenas prácticas	Te muestra cómo escribir tests profesionales (estructura, nombres, etc.).

1.2 — Idea clave del módulo

Completa esta frase que vimos al inicio de la sesión:

"La IA acelera la creación de tests, mejora la cobertura y ayuda a detectar casos límite que se nos escapa y no solemos contemplar."

¿Qué **NO** sustituye la IA en el proceso de testing? Explícalo con tus propias palabras:

Mi criterio. Ella me puede hacer el código de los test y yo lo revisaré y si estoy de acuerdo lo acepto y lo utilizo si no, puedo pedir que me genere otro con ciertos parámetros.
Ese criterio no podría tenerlo ella, de decirle que va y que no...

BLOQUE 2 — Tipos de pruebas que genera la IA (1.2)

Para cada elemento de la columna central, escribe si pertenece a **U** (Unitaria) o **I** (Integración) y explica brevemente por qué:

U / I	Qué se prueba	¿Por qué es unitaria o de integración?
U	Una función que calcula el total de un carrito	Probamos una función.

I	El flujo completo de login → compra → logout	Seguimos el flujo completo del sistema.
U	Un método de una clase que valida un email	Logica aislada.
I	La interacción entre el endpoint /usuarios y la base de datos	Tenemos interacciones entre sistemas.
U	Una función que convierte Celsius a Fahrenheit	Es una función simple.
I	Un endpoint REST GET /products que devuelve JSON	API completa.
U	El constructor de una clase con sus propiedades	Es una función de clase con propiedades dentro.
I	Que un módulo de pagos y un módulo de pedidos se comuniquen bien	Tenemos comunicación entre módulos.

2.2 — Definiciones

Escribe con tus propias palabras qué diferencia a cada tipo de prueba. Incluye al menos un ejemplo de uso real:

Tipo de prueba	Definición con tus palabras
Prueba unitaria	Es aquella que prueba el código de forma aislada, es decir prueba partes pequeñas del código para ver su comportamiento.

Prueba de integración

Esta es lo contrario, la usaremos para hacer pruebas de como interactúan varias partes de sistema entre si.

BLOQUE 3 — Frameworks que la IA domina bien (1.3)

Une cada framework con su lenguaje principal y escribe para qué se usa:

Framework	Lenguaje principal	¿Para qué se usa principalmente?
Jest	JavaScript	Testing de frontend/backend JS
pytest	Python	Tests unitarios y profesionales
JUnit	Java	Testing en Java
PHPUnit	PHP	Testing en PHP
RSpec	Ruby	Testing estilo BDD
Vitest	JavaScript	Alternativa moderna a Jest
xUnit	C#	Testing en .NET

3.2 — pytest vs unittest: Verdadero o Falso

Marca **V** (Verdadero) o **F** (Falso) y justifica brevemente las que sean Falsas:

V / F	Afirmación
V	pytest tiene una sintaxis más sencilla que unittest: solo usa assert en lugar de self.assertEqual(). EXPLICACION: En pytest puedes escribir tests como funciones normales y usar directamente assert, lo que hace el código más limpio y legible. Y no necesitamos heredar nada .
F	unittest necesita menos código (boilerplate) que pytest para definir un test. EXPLICACION: En realidad, es al revés unittest suele requerir más código "boilerplate" que pytest para definir un test.
V	pytest permite parametrizar tests con @pytest.mark.parametrize. EXPLICACION: permite ejecutar una misma función de prueba con diferentes conjuntos de datos, generando un caso de prueba independiente para cada combinación.
F	Si heredas un proyecto unittest, no puedes usar pytest sin reescribir todo el código. EXPLICACION: Si puede (Compatible)
V	pytest tiene más de 1000 plugins disponibles, como pytest-cov para medir cobertura. EXPLICACION: pytest-cov es uno de los más importantes para cobertura
V	Para proyectos legacy corporativos donde no puedes instalar dependencias externas, unittest puede ser adecuado EXPLICACION: es adecuado (y a veces la única opción viable) Especialmente en entornos cerrados, regulados o legacy
V	La IA conoce perfectamente pytest y puede generar tests con él sin problema. EXPLICACION: SI, porque es la mas estandar.

BLOQUE 4 — Estructura del prompt para testing con IA (1.4)

Un prompt profesional para generar tests debe incluir **6 elementos clave**. Escríbelos y explica para qué sirve cada uno:

1	Función o módulo	Qué quieres testear
2	Framework	pytest, Jest, etc.
3	Casos edge	Límites (0, valores grandes...)
4	Mocks	Simular dependencias
5	Formato de salida	Cómo quieres los tests
6	Explicación (opcional)	Para entender cada test

4.2 —Estrategia de 3 Niveles: ¿qué nivel le corresponde?

Clasifica cada prompt en su nivel (1 = Funciones simples · 2 = Lógica compleja · 3 = Sistemas completos) y razona por qué:

Prompt	Nivel	Explicación
Capitalizar texto	1	Función simple
Sistema caché distribuido	3	Sistema complejo
Parser matemático	2	Lógica compleja
Filtrar pares	1	Función simple
Sistema de permisos	3	Sistema completo
Celsius a Fahrenheit	1	Función básica

4.3 — Tipos de casos de prueba en el prompt

Cuando pedimos tests a la IA, vimos 3 tipos de casos. Clasifica cada ejemplo: **N** (Normal), **L** (Límite) o **E** (Error):

N / L / E	Caso de prueba	¿Por qué es de ese tipo?
N	<code>dividir(10, 2)</code> → se espera 5.0	CASO NORMAL
E	<code>dividir(10, 0)</code> → debe lanzar <code>ZeroDivisionError</code>	ERROR
L	<code>dividir(0, 5)</code> → se espera 0.0	LIMITE
E	<code>dividir("diez", 2)</code> → entrada inválida	TIPO INCORRECTO
L	<code>dividir(1e100, 10)</code> → número muy grande	VALOR EXTREMO
N	<code>dividir(-10, -2)</code> → ambos negativos dan positivo	CASO VALIDO

E	dividir(None, 5) → valor nulo como entrada	ERROR
N	dividir(5, 2) → resultado con decimales (2.5)	NORMAL

BLOQUE 5 — Pregunta de síntesis

Un compañero/a dice: *"Si la IA genera los tests automáticamente, no necesito revisarlos."* ¿Estás de acuerdo? Razona tu respuesta con al menos **dos argumentos concretos** apoyándote en lo visto en los 4 bloques de este ejercicio:

NO ESTOY DE ACUERDO.

Porque puede cometer errores al generar los test, no conoce el contexto global del proyecto, no tiene ni idea de los requisitos reales que necesitamos. En fin no porque la IA es una ayuda como lo acabo de hacer yo con esta tarea, pero algunas cosa la eh buscando fuera de la IA para corroborar que lo que ella me dice es cierto. Entonces tiene doble informacion una debe ser analizada por ti y la otra ya esta resumida por IA entonces aqui es donde entra tu criterio y saca tu propio analisis.